

Lecture 4: Data Preparation

PSTAT100: Data Science - Concepts and Analysis

John Inston 

johninston@ucsb.edu

University of California, Santa Barbara

April 9, 2026

Overview

Aims of the lecture

- Study common data problems.
- Handling missing data.
- Searching for duplicate data.
- Fixing inconsistent data types.
- Labelling issues.

Required Libraries

In this lecture we will be using the following libraries:

```
1 import pandas as pd
2 import numpy as np
3 import random as rd
```

Tidy Data Review

Tidy Data Standard

Tidy Data Standard

For data to be tidy, it must satisfy the following three rules:

1. Each variable is a column.
2. Each observation is a row.
3. Each type of observational unit forms a table.

- If only it were so easy...

Other Common Data Problems

- Missing values.
- Duplicate records.
- Inconsistent or incorrect data types.
 - Dates, numbers as strings etc.
- Invalid values / outliers.
- Labelling issues.

Data Preparation

Data Preparation

What is data preparation?

Data Preparation

Data preparation is the process of cleaning and transforming data to make it suitable for analysis.

- When we load data from a file we take steps to facilitate analysis.
- Typically this process involves:
 -  **Identifying** what is wrong with the data.
 -  **Determining** how to “fix” the problem.
 -  **Updating** and **checking** the result.

Missing Data

What causes missing data?

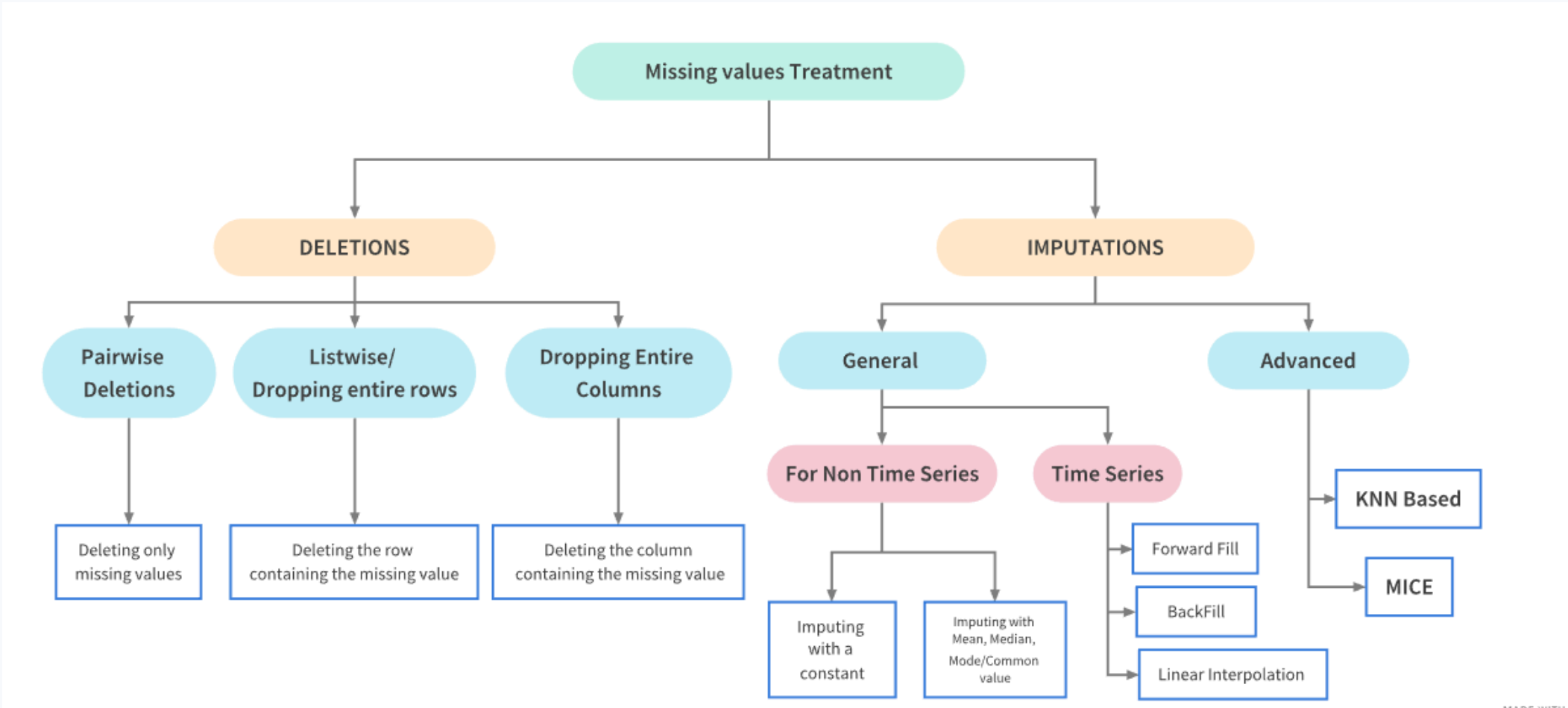
- Missing data can arise for a variety of reasons.
 - Field was **never collected**.
 - Data was collected but **lost** or **destroyed**.
 - Data was collected but **recorded incorrectly**.
- Understanding the **cause** of missing data can be challenging. 🤔
 - But it can help us make good decisions! 👍

How do we handle missing data?

- Two approaches:
 - **Deletion**: Removing the missing data from the dataset.
 - **Imputation**: Filling in the missing data with replacement values.

Missing Data Treatments

Treatment Dependencies



Missing Data

Libraries and Practice Data

Library

To graphically analyze the missingness of data we can use the [Missingno Library](#), which provides a suite of tools for visualizing missing data.

```
1 import missingno as msno
```

Practice Data

For our example we consider a simple example of a dataset with missing data. We load the `nfl.csv` dataset which contains detailed NFL play-by-play data from 2009.

```
1 nfl_data = pd.read_csv('data/nfl.csv')
```


Inspecting the Data

Let's take a first look at the first few rows and the shape.

```
1 nfl_data.head()
```

NFL Play-by-Play Data

	Date	GameID	Drive	qtr	down	time	TimeUnder	TimeSecs	PlayTimeDiff	SideofField	...	ya
0	2009-09-10	2009091000	1	1	NaN	15:00	15	3600.0	0.0	TEN	...	
1	2009-09-10	2009091000	1	1	1.0	14:53	15	3593.0	7.0	PIT	...	1.14
2	2009-09-10	2009091000	1	1	2.0	14:16	15	3556.0	37.0	PIT	...	
3	2009-09-10	2009091000	1	1	3.0	13:35	14	3515.0	41.0	PIT	...	-5.03
4	2009-09-10	2009091000	1	1	4.0	13:27	14	3507.0	8.0	PIT	...	

5 rows × 102 columns

- We can see that the dataset contains some missing data - i.e. NaN values.

```
1 nfl_data.shape
```

```
(362447, 102)
```

- We have 102 columns and ~360,000 rows!

Quantifying Missingness

Counting Missing Values

- We start by counting the number of missing values in each column.
 - `isnull` identifies missing values and `sum` counts them.

```
1 missing_values_count = nfl_data.isnull().sum()  
2 missing_values_count.sort_values(  
3     ascending=False  
4     ).head(10)
```

```
DefTwoPoint      362433  
BlockingPlayer   362341  
TwoPointConv     361919  
ChalReplayResult 359476  
RecFumbPlayer    358513  
RecFumbTeam      358513  
Interceptor      358387  
FieldGoalDistance 354528  
FieldGoalResult  354431  
ExPointResult    353399  
dtype: int64
```

Percentage Missingness

```
1 # how many total missing values do we have?  
2 total_cells = np.prod(nfl_data.shape)  
3 total_missing = missing_values_count.sum()  
4  
5 # percent of data that is missing  
6 percent_missing = (total_missing/total_cells) * 100  
7 print(percent_missing)
```

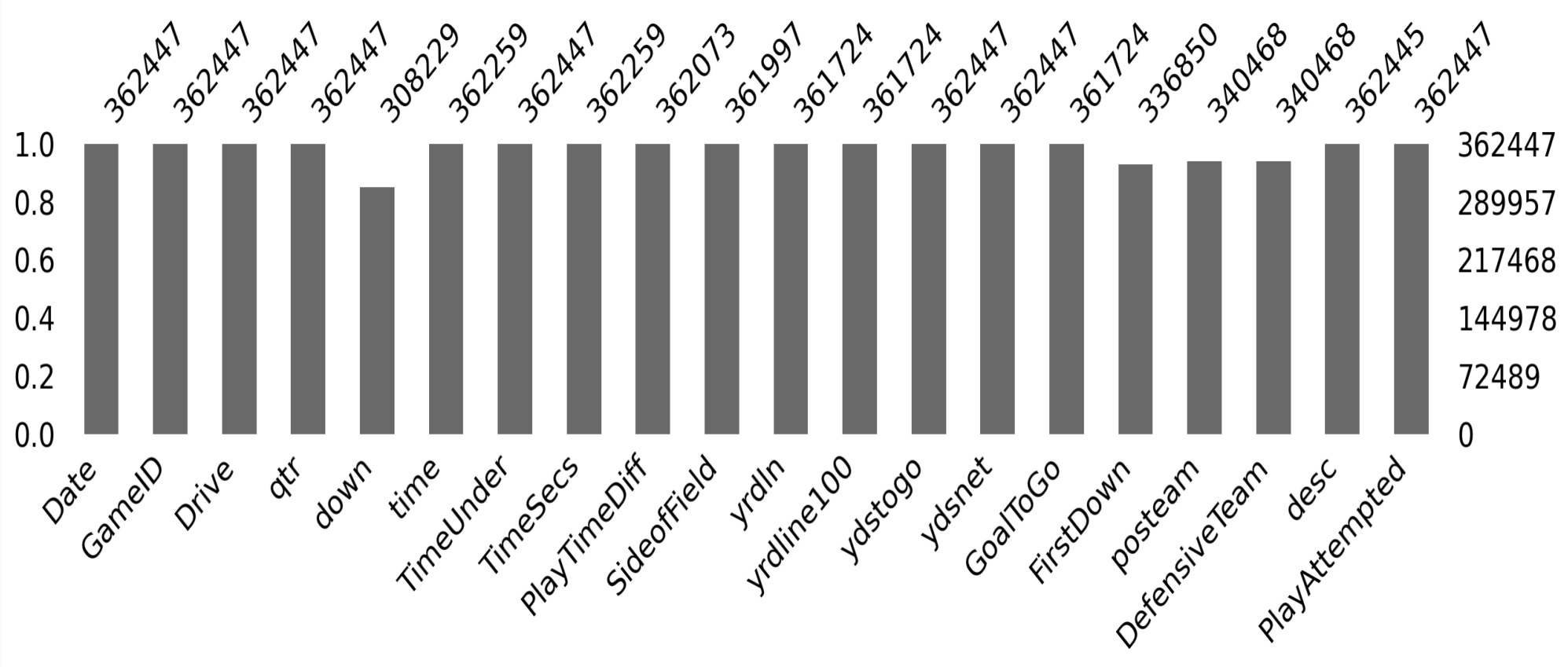
```
27.652267428200588
```

- There are many nearly empty columns.

Missing Data Bar Chart

- We can use `msno.bar` to visualize the missing data counts (for the first few columns to make it easier to see).

```
1 msno.bar(nfl_data.iloc[:, 0:20], figsize=(15, 2.5))
```

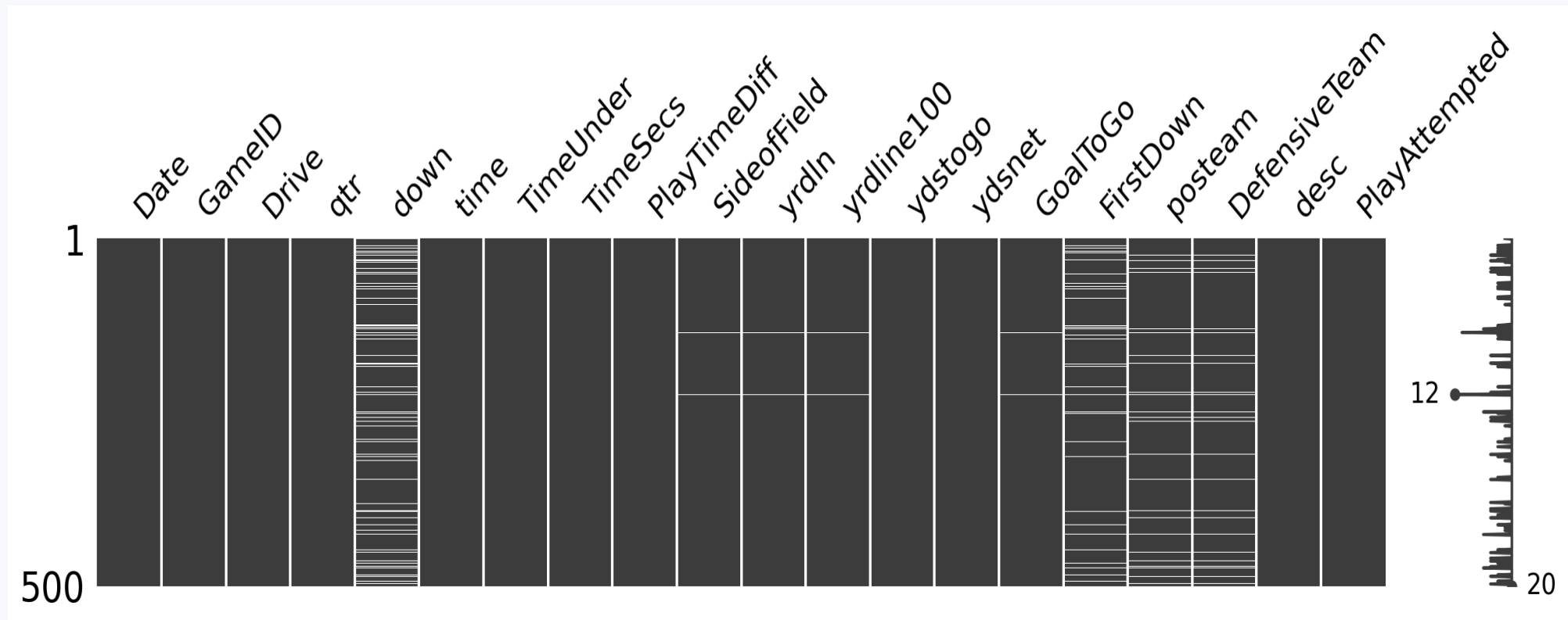


Missing Data Bar Chart

Missing Data Matrix

- `msno.matrix` visualizes the missing data (white = missingness) for 500 random rows.

```
1 np.random.seed(42) # set seed for reproducible sample
2 nfl_data_sample = nfl_data.sample(500)
3 msno.matrix(nfl_data_sample.iloc[:,0:20], figsize=(15, 3))
```

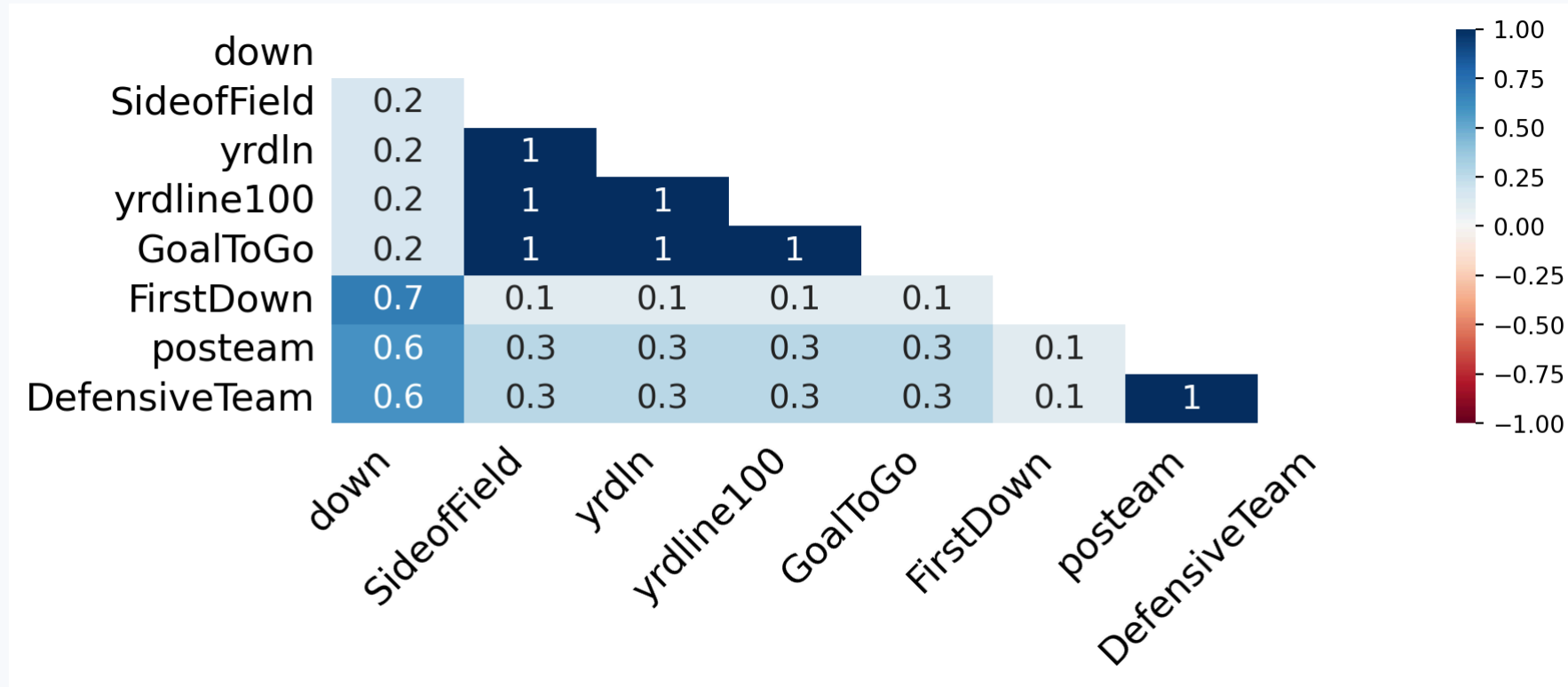


- We see that the missing data patterns among columns are quite similar.
 - This indicates underlying relationships between the variables.

Missing Data Heatmap

- `msno.heatmap` visualizes these relationships.

```
1 msno.heatmap(nfl_data_sample.iloc[:,0:20], figsize=(10, 3))
```

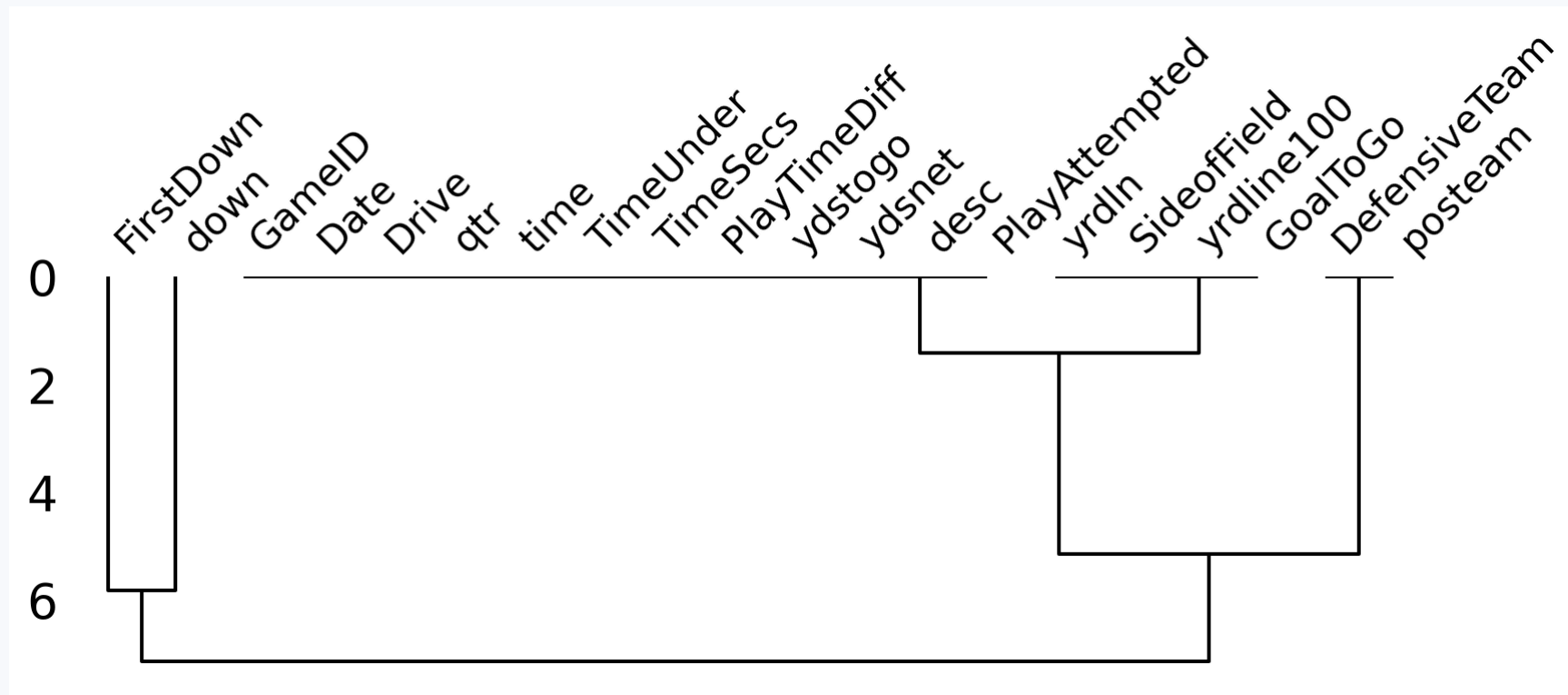


- Positive correlation indicates variables are often missing together.
- Negative correlation indicates variables are often missing opposite each other.

Missing Data Dendrogram

- We can use `msno.dendrogram` to visualize the hierarchical relationships between missing data.

```
1 msno.dendrogram(nfl_data_sample.iloc[:,0:20], figsize=(10, 3))
```



- Cluster leaves linked at a distance of 0 fully predict one another's presence.
- It is clear then that this data set's missingness is not **completely random**.

Understanding the Plots

Lots of helpful information for analysis

- Variables with missing data can cause problems for analysis.
- How are the variables related to each other in terms of missingness.
 - Which variables are missing together.
 - Which variables are candidates for deletion / imputation.

Missingness is itself information

- In real data missingness often carries signal.
 - **Example 1:** Income left blank because the person doesn't want to disclose it.
 - **Example 2:** Missing follow up appointments in drug trial because patients became too ill.
- By understanding missingness we help to avoid bias in our analysis.
 - **Example 1:** Sample will be mostly low and middle income earners.
 - **Example 2:** Sample will be mostly patients who responded well to the treatment.

Why is the Data Missing?

Ask the right questions!

- Is this data missing because it was never collected or because it doesn't exist?
 - If the data is missing because it doesn't exist (e.g. the age of pet for someone who doesn't have a pet) then we shouldn't impute the data.
 - If the data is missing because it was never collected then we should try imputation.

Missing Data Classifications

1. **Missing Completely at Random (MCAR)** - missingness is completely random and independent of the observed data.
2. **Missing at Random (MAR)** - missingness is not random, but can be fully accounted for by variables where there is complete information.
3. **Missing Not at Random (MNAR)** - missingness depends on unobserved data or the value of the missing data itself.

Deleting Missing Data

What is deletion?

- When data is MCAR we can consider **deletion**, the removing missing data from the dataset.
 - **Pro:** This is a quick and effective way to handle missing data.
 - **Con:** It loses information.

Deleting Rows with Missing Data

```
1 nfl_data_delete_rows = nfl_data.dropna()  
2 # just how much data did we lose?  
3 print("Rows in original dataset: %d \n" % nfl_data.shape[0])  
4 print("Rows with na's dropped: %d" % nfl_data_delete_rows.shape[0])
```

Rows in original dataset: 362447

Rows with na's dropped: 0

- We have deleted every row!

Deleting Columns with Missing Data

```
1 nfl_data_delete_columns = nfl_data.dropna(axis=1)  
2 # just how much data did we lose?  
3 print("Columns in original dataset: %d \n" % nfl_data.shape[1])  
4 print("Columns with na's dropped: %d" % nfl_data_delete_columns.shape[1])
```

Columns in original dataset: 102

Columns with na's dropped: 37

- We have deleted 37 out of 102 columns (approximately 36% of the data).

Imputation

What is imputation?

- Imputation is the process of replacing missing data.
 - Methods depend on data characteristics.
 - What might be a good candidate value for missing data?

When to use imputation?

- Looking at our example data we see that `TimeSecs` is a column that is missing a large number of values.
 - Looking at the [documentation](#) we see that this column is the number of seconds left in the game when the play occurred.
 - It is likely that this data was not recorded therefore we can use imputation.
- However, `PenalizedTeam` is also missing values.
 - This column is the team that was penalized.
 - This data is missing because there was no penalty therefore we should not impute the data.

Simple Imputation Methods

`fillna` method

- In pandas we can use the `fillna` method to impute missing data.
 - We can specify a value to replace missing data with.
 - Some **constant value**.
 - **Statistics** such as the mean, median, or mode.
 - Alternatively we can specify:
 - `method` to specify:
 - `ffill` - forward fill (use the previous value).
 - `bfill` - backward fill (use the next value).
 - `axis` to specify the axis to impute along (i.e. rows or columns).

Imputation is a powerful tool and a topic in it's own right. We will not cover it in depth in this course but recommend you explore it further if your project data has missing values.

Imputation Example

```
1 nfl_data_imputed = nfl_data.bfill(axis=0).fillna(0)
2 nfl_data_imputed.head()
```

	Date	GameID	Drive	qtr	down	time	TimeUnder	TimeSecs	PlayTimeDiff	SideofField	...	ya
0	2009-09-10	2009091000	1	1	1.0	15:00	15	3600.0	0.0	TEN	...	1.14
1	2009-09-10	2009091000	1	1	1.0	14:53	15	3593.0	7.0	PIT	...	1.14
2	2009-09-10	2009091000	1	1	2.0	14:16	15	3556.0	37.0	PIT	...	-5.03
3	2009-09-10	2009091000	1	1	3.0	13:35	14	3515.0	41.0	PIT	...	-5.03
4	2009-09-10	2009091000	1	1	4.0	13:27	14	3507.0	8.0	PIT	...	0.16

5 rows × 102 columns



Duplicate Values

Identifying Duplicate Values

- Well structured data should not have **duplicate values**.
 - Typically observations have a **unique identifier**.
 - If there are duplicate values, we need to identify them and decide how to handle them.

Example Data

For our example we consider synthetic customer data `customer.csv`.

Duplicate Data Example

```
1 customer_data = pd.read_csv('data/customer.csv')
2 customer_data.head()
```

Customer Data

	customer_id	name	email	age	city	spend
0	101	Alice Johnson	alice@email.com	34	New York	250.00
1	102	Bob Smith	bob@email.com	28	Chicago	180.50
2	103	Carol White	carol@email.com	45	Houston	320.00
3	101	Alice Johnson	alice@email.com	34	New York	250.00
4	104	David Brown	david@email.com	52	Phoenix	410.75

- We can see that the dataset contains some duplicate values, lets count them using

`customer_data.duplicated()`:

```
1 dup_total = customer_data.duplicated().sum()
2 dup_index = customer_data.loc[customer_data.duplicated()]
3 print(f"Total duplicate values: {dup_total}")
4 print(f"Duplicate index values: {dup_index}")
```

```
Total duplicate values: 6
Duplicate index values: Index([3, 5, 10, 12, 14, 16], dtype='int64')
```

- Is this the whole story? Lets see..

Near Duplicates

Near Duplicates

- We have identified 6 exact duplicates but have we missed any near duplicates?
- A common source of near duplicates is string formatting inconsistency - `Carol White` vs `carol white`.

Make Formatting Consistent

- Let's make the formatting across all columns containing strings by:
 - Converting first letters of names to uppercase with `str.title()`.
 - Removing leading and trailing whitespace with `str.strip()`.

```
1 customer_data['name'] = customer_data['name'].str.strip().str.title()
2 customer_data['email'] = customer_data['email'].str.strip().str.lower()
3 customer_data['city'] = customer_data['city'].str.strip().str.title()
```

- Now if we once again count the duplicates we find that:

```
1 dup_total = customer_data.duplicated().sum()
2 dup_index = customer_data.loc[customer_data.duplicated()]
3 print(f"Total duplicate values: {dup_total}")
4 print(f"Duplicate index values: {dup_index}")
```

```
Total duplicate values: 7
Duplicate index values: Index([3, 5, 8, 10, 12, 14, 16],
dtype='int64')
```


Dealing with Duplicate Values

Deleting Duplicate Values

- Most often we will simply **delete** duplicate values.
 - We can use `drop_duplicates`.

```
1 customer_data_no_dup = customer_data.drop_duplicates()  
2 customer_data_no_dup.duplicated().sum()
```

```
np.int64(0)
```

Aggregating Duplicate Values

- Sometimes we may want to **aggregate** duplicate values (to maximize information retention).
 - We can use `groupby` to aggregate duplicate values.

```
1 customer_data_agg = customer_data.groupby(  
2     'customer_id'  
3     ).agg({  
4         'name': 'first',  
5         'email': 'first',  
6         'age': 'mean',  
7         'city': 'first',  
8         'spend': 'sum'  
9     })  
10 customer_data_agg.duplicated().sum()
```

```
np.int64(0)
```

Inconsistent Data Types

Why types matter

- Pandas **infers** types when you load a file
 - This inference is not always correct.
 - Wrong types break operations:
 - **Averaging strings.**
 - **Comparing dates stored as text.**
- **Goal:** each column should have a type that matches its **meaning**.

Spotting and fixing

- Use available **data dictionaries** or domain knowledge to help identify the correct type for each column.
- **Inspect** with `dtypes`, `info()` and **spot-check** values with `head()`, `unique()`.
- **Coerce** into correct types (`pd.to_numeric`, `pd.to_datetime`)
- When reading CSVs, you can steer types with `dtype={...}` or parse dates with `parse_dates=[...]`.

Examples of Inconsistent Data Types

```
1 example_df = pd.DataFrame({
2     "Dates":["2024-01-01", "2024-01-02", "2024-01-03"],
3     "Floats":["1.1", "2.2", "3.3"],
4     "Booleans":["True", "False", "True"],
5 })
```

Dates as strings

```
1 print(f"Date column original type: {example_df['Dates'].dtype}")
2 example_df['Dates'] = pd.to_datetime(example_df['Dates'])
3 print(f"Date column new type: {example_df['Dates'].dtype}")
```

```
Date column original type: str
Date column new type: datetime64[us]
```

Numbers are strings

```
1 print(f"Floats column original type: {example_df['Floats'].dtype}")
2 example_df["Floats"] = pd.to_numeric(example_df["Floats"])
3 print(f"Floats column new type: {example_df['Floats'].dtype}")
```

```
Floats column original type: str
Floats column new type: float64
```

Booleans are strings

```
1 print(f"Booleans column original type: {example_df['Booleans'].dtype}")
2 example_df["Booleans"] = (
3     example_df["Booleans"].str.strip().str.lower().map({"true": True, "false": False}).astype("boolean")
4 )
5 print(f"Booleans column new type: {example_df['Booleans'].dtype}")
```

```
Booleans column original type: str
Booleans column new type: boolean
```

Mixed Data Formats

Extra Problems

- Hand prepared data often contains mixed data formats.
 - Dates written as strings in different formats.
 - Names with titles, middle initials etc.
 - The options are endless...
- We will not cover this in depth in this course but recommend you explore it further if your project data has mixed data formats.

Harder Example

```
1 example_df = pd.DataFrame({  
2     "Dates":["2024-01-01", "21-12-2001", "December 21, 2001", "01/02/2024", "2018-03-04"],  
3     "Booleans":["True", "false", "true", "no", "yes"],  
4     })
```

Examples of Mixed Data Formats

Date Formats

```
1 example_df['Dates'] = pd.to_datetime(example_df['Dates'], format="mixed")
2 print(example_df['Dates'])

0    2024-01-01
1    2001-12-21
2    2001-12-21
3    2024-01-02
4    2018-03-04
Name: Dates, dtype: datetime64[us]
```

Boolean Formats

```
1 example_df['Booleans'] = example_df['Booleans'].str.strip(
2 ).str.lower(
3 ).map({
4     "true": True, "false": False, "yes": True, "no": False,
5     }).astype("boolean")
6 print(example_df['Booleans'])

0    True
1   False
2    True
3   False
4    True
Name: Booleans, dtype: boolean
```

- Pandas is very capable but often requires looking up syntax!

Invalid Values

Wrong but present

- **Invalid values** are not missing: they appear as real entries but lie outside allowed rules:
 - Impossible ages.
 - Negative counts.
- They differ from **outliers**: outliers can be legitimate extremes; invalid values break domain logic.
- Can be hard to spot since they will typically not produce errors.

What to do

- Define **rules** from the data dictionary or domain (ranges, allowed categories, regex for IDs).
- Use boolean masks: `Series.between`, or `isin` to flag rows
- Drop, replace, or set to `NaN` and treat as missing.
- Document every rule: “why is this invalid?” matters as much as the code.

```
1 ages = pd.Series([34, 200, 28, -1, 45])
2 ages[(ages < 0) | (ages > 120)]
```

```
1    200
3     -1
dtype: int64
```

Labelling Issues

Same concept, different labels

- Categories may be spelled differently, use different casing, or mix codes (`"NYC"` vs `"nyc"` vs `"New York"`).
- Column names can be inconsistent across files (`customerID` vs `customer_id`), which blocks merges.

Cleaning labels

- Normalize text: `str.strip` , `str.lower` / `str.title` , remove extra spaces.
- Build a **mapping dictionary** from raw labels to canonical ones and use `map` or `replace` .
- For analysis, prefer a single **canonical vocabulary** per variable; keep a small “codebook” you can reuse.

```
1 city_raw = pd.Series([" nyc ", "NYC", "New York", "new y  
2 city_raw.str.strip().str.title().replace({"Nyc": "New Yor
```

```
0    New York  
1    New York  
2    New York  
3    New York  
dtype: str
```

Parsing Dates

Dates are easy to get wrong

- Stored as strings with ambiguous order (`01/02/2024`), mixed formats, or Excel serial numbers.
- Wrong parsing silently shifts days or produces `NaT` only where you are not looking.

Tools in pandas

- At load time: `pd.read_csv(..., parse_dates=["date_col"])` when the column is consistently formatted.
- After load: `pd.to_datetime`; use `format='...'` when you know the pattern, and `errors='coerce'` to isolate bad strings.
- Use `dayfirst=True` or `yearfirst=True` when the data is not ISO-8601 and locale order matters.

```
1 # Day-first strings (common outside the US); invalid entries
2 pd.to_datetime(
3     pd.Series(["15/03/2024", "02/04/2024", "not a date"])
4     errors="coerce",
5     dayfirst=True,
6 )
```

```
0    2024-03-15
1    2024-04-02
2             NaT
dtype: datetime64[us]
```


Data Preparation Summary

Things to remember

- Make sure you thoroughly understand the data you are working with.
 - Read the documentation / dictionaries.
 - Check for the common problems.
 - Be prepared to go back and fix things you have missed.
- Document your data preparation process.
 - Highlight the issues you have found.
 - Justify your decisions.
- Being thorough initially will save you time in the long run!

Data Preparation Procedural Guide

How to clean, a general guide!

1. Inspection.
2. Variable labeling.
3. Remove redundant / unused variables.
4. Fix data types and mixed formats.
5. Handle duplicates.
6. Handle missing values.
7. Clean strings / encode categorical variables.
8. Handle outliers / invalid values.
9. Final validation and documentation.

This is a good guide to follow but it is not a strict recipe (as always)!



Nice and clean!

Conclusion

What we covered

- Data preparation:
 - Missing data.
 - Duplicates.
 - Inconsistent data types.
 - Mixed data formats.
 - Invalid values.
 - Labelling issues.
 - Parsing dates.

What's next?

- Visualizations in python.
 - Matplotlib.
 - Seaborn.
 - Plotly.
 - Altair.

References